



REPUBLIQUE DU CAMEROUN
Paix – Travail – Patrie

UNIVERSITE DE DOUALA

ECOLE NATIONALE SUPERIEURE
POLYTECHNIQUE DE DOUALA

B.P. 2701 Douala
Tél. (237) 697 542 240
Site web : www.enspd-udo.cm

REPUBLIC OF CAMEROON
Peace – Work – Fatherland

THE UNIVERSITY OF DOUALA

NATIONAL HIGHER POLYTECHNIC
SCHOOL OF DOUALA

P.O.Box:2701 Douala
Phone : (237) 697 542 240
Email: contact@enspd-udo.cm



MÉMOIRE DE PROJET TUTORÉ

Filière : GIT (Génie Informatique & Télécommunication)
GLo (Génie Logiciel) III

Thème : OBSERVER DESIGN PATTERN

*Etude et Application du Patron Observateur dans une
messagerie instantanée : Cas de Mporôl.*

Contributeurs

NGOOH Agnès Serena

NDOUNGUE Essoh Abel Thomas

ATEKOUMBO Excel Dexte Joris

Sous la supervision de

Dr. TOTTO

Année Académique : 2024/2025

DEDICACE

*À nos chers frères et sœurs,
Nos chers camarades et amis,
Nos chers parents et tuteurs,
Nos chers futurs partenaires,
Nos chers instituteurs,
Notre chers Responsable Pédagogique,
Monsieur le Directeur de l'ENSPD.*

Table des matières

DEDICACE.....	2
Table des matières	3
Résumé exécutif	4
Objectifs du rapport.....	5
Remerciements	6
I. Introduction - Contexte et définition des design patterns.....	7
I-1. Introduction au concept de l'Observer Pattern / Patron Observateur (en français)....	8
II. Analyse SWOT et FAST du projet	11
1. Analyse SWOT (diagramme).....	11
2. Analyse FAST (diagramme)	12
III. Application de l'Observer Design Pattern dans une Messagerie Instantanée	13
1. Contexte et justification	13
2. Structure et implémentation dans une application JavaScript.....	13
3. Bénéfices pour une messagerie instantanée	13
4. Exemple d'implémentation en JavaScript.....	13
IV. Mporôl.....	15
IV-I. Architecture de l'application	15
Composants clés :	16
IV-II. Fonctionnalités de Mporôl	17
IV-III Implémentation du Observer Design Pattern.....	17
IV-IV Optimisation et amélioration	20
Conclusion.....	21
Références	22
Figure 1 - code source observateur.....	17
Figure 2 - mise en évidence de la mise à jour	18
Figure 3 - mise en évidence des notifications	19

Résumé exécutif

L'Observer Design Pattern est un patron de conception comportemental clé dans le développement logiciel, permettant d'établir une relation un-à-plusieurs entre un objet central (le Sujet ou Observable) et plusieurs Observateurs. Ce modèle facilite la synchronisation des objets sans nécessiter d'interactions directes, offrant ainsi une communication efficace entre les composants d'une application.

En permettant aux observateurs de s'inscrire pour recevoir des notifications sur les changements d'état du sujet, l'Observer Design Pattern favorise un faible couplage entre les composants. Cela permet aux développeurs d'ajouter ou de retirer des observateurs facilement, améliorant ainsi la maintenabilité et l'évolution des applications.

Ce modèle est particulièrement pertinent dans les systèmes de notifications en temps réel, comme les applications de messagerie instantanée, et dans le développement d'interfaces utilisateur réactives. Par exemple, dans une application de messagerie instantanée développée en Javascript, chaque utilisateur d'un groupe de discussion agit comme un observateur. Lorsqu'un message est envoyé, tous les membres sont automatiquement notifiés, leur permettant d'afficher instantanément le nouveau message sans actualiser l'interface.

À travers cela, on en conclut que l'Observer Design Pattern est essentiel pour créer des applications dynamiques et réactives, enrichissant l'expérience utilisateur tout en simplifiant la gestion du code.

Objectifs du rapport

Ce rapport a pour objectif d'**analyser en profondeur le fonctionnement du Observer Design Pattern**, en mettant en lumière ses principes fondamentaux, ses mécanismes de communication et sa structure logicielle. Il permettra d'illustrer **ses applications pratiques**, notamment dans des environnements nécessitant une mise à jour automatique et dynamique des informations, comme les **systèmes de messagerie instantanée**. Dans ce contexte, l'**Observer Pattern** joue un rôle essentiel pour gérer les notifications en temps réel, où chaque utilisateur doit recevoir instantanément les nouveaux messages sans effectuer de mise à jour manuelle. Cette étude approfondie se concentrera sur **l'implémentation de ce modèle** dans un programme développé en **Javascript**, en mettant en avant ses avantages en termes de flexibilité et de gestion des événements. Enfin, ce rapport servira de **guide méthodologique**, permettant une intégration efficace de l'**Observer Pattern** dans l'application et garantissant une meilleure gestion des interactions entre les composants logiciels.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet sur l'**Observer Design Pattern**.

Tout d'abord, un immense merci à nos **enseignants et mentors**, dont les conseils avisés ont enrichi notre compréhension des **design patterns** et de leur application dans le développement logiciel. Leur expertise et leur disponibilité ont été des atouts précieux tout au long de cette étude.

Nous remercions également nos **collègues et pairs**, avec qui nous avons pu échanger des idées, affiner notre réflexion et tester concrètement l'implémentation de l'**Observer Pattern** dans une application de **messagerie instantanée en Javascript**. Leurs retours et suggestions ont été essentiels pour l'amélioration de ce travail.

Enfin, nous souhaitons exprimer notre reconnaissance aux **auteurs et chercheurs**, dont les travaux ont servi de référence et ont enrichi cette analyse avec des concepts théoriques solides et des exemples pratiques. Ce rapport est le fruit d'une collaboration et d'une exploration approfondie, et nous sommes honorés d'avoir pu compter sur autant de contributions précieuses pour mener à bien ce projet.

I. Introduction - Contexte et définition des design patterns

Les **design patterns**, ou motifs de conception, transcendent le simple code pour représenter des solutions éprouvées et optimisées à des problèmes récurrents en développement logiciel. On peut les comparer à des recettes de cuisine : en suivant une recette bien établie, vous êtes assuré d'obtenir un plat savoureux et bien présenté, sans avoir à improviser à chaque étape. De la même manière, les design patterns fournissent **un cadre standardisé** qui permet aux développeurs de résoudre efficacement des problèmes complexes, tout en minimisant les erreurs courantes et en accélérant le processus de développement.

Parmi les nombreux design patterns, les 23 modèles classiques présentés par le « *Gang of Four* », [*Design Patterns – Elements of Reusable Object-Oriented Software*¹](#), » sont souvent considérés comme fondamentaux. Ces modèles se divisent en trois catégories principales :

1. Modèles créatifs : ce sont ces modèles qui créent des objets, plutôt que d'avoir à instancier des objets directement. Cela donne au programme plus de flexibilité pour décider quels objets doivent être créés dans un cas donné.

2. Modèles structurels : concernent la composition de classes et d'objets. Ils utilisent l'héritage pour composer des interfaces et définissent des façons de composer des objets afin d'obtenir de nouvelles fonctionnalités.

3. Modèles comportementaux : Ils se concentrent sur la manière dont les objets interagissent et communiquent entre eux.

L'adoption de ces design patterns contribue à la clarté et à la robustesse du code. Lorsqu'une équipe de développeurs collabore sur un projet ou qu'un développeur reprend un code existant, la présence de motifs de conception bien établis facilite la compréhension de l'architecture du système. De plus, ces modèles favorisent une certaine flexibilité ; en intégrant dès le départ une modularité réfléchie, il devient plus aisé d'ajouter de nouvelles fonctionnalités ou de modifier des parties spécifiques sans perturber l'ensemble du logiciel.

Enfin, cette approche structurée simplifie également la maintenance du code. En évitant de repartir de zéro à chaque modification, les développeurs peuvent se concentrer sur l'amélioration continue et l'évolution du produit, tout en réduisant le risque d'introduction de nouveaux bogues. En somme, les design patterns ne sont pas seulement des outils techniques ; ils représentent une philosophie de développement qui valorise la collaboration, la réutilisation et l'efficacité.

Parmi eux, **l'Observer Pattern** est un modèle qui facilite la communication entre objets sans qu'ils soient directement dépendants les uns des autres.

I-1. Introduction au concept de l'Observer Pattern / Patron Observateur (en français)

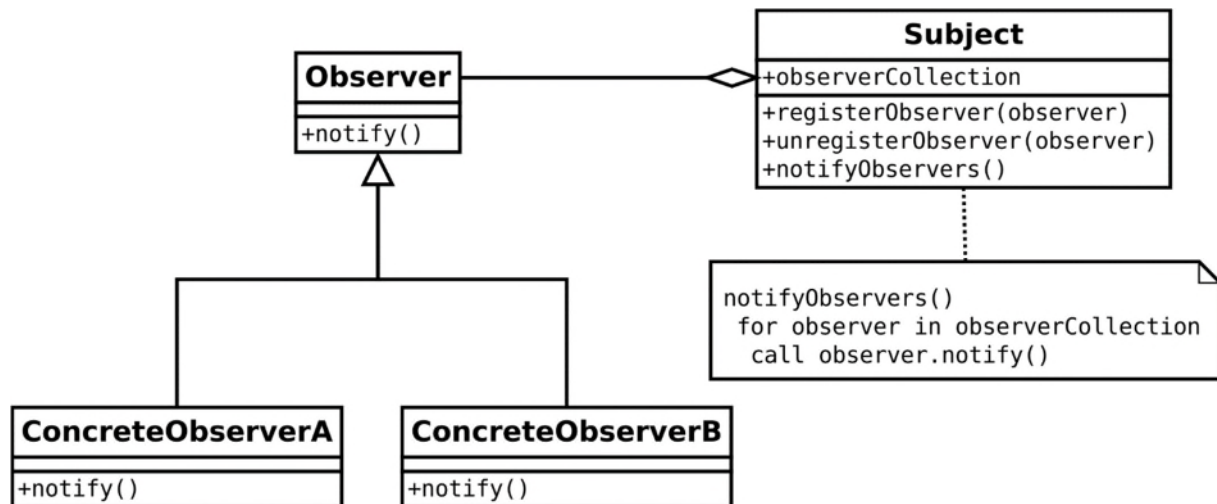
En génie logiciel, le **patron observateur** est un patron de conception de la famille des **patrons (modèles) comportementaux**. Ce pattern repose sur une structure où un **sujet** (Observable) maintient une liste d'**observateurs**, qui sont automatiquement mis à jour lorsqu'un changement se produit. Cette approche est particulièrement utile dans les systèmes où plusieurs éléments doivent réagir simultanément à une modification, comme dans les **interfaces utilisateur**, les **systèmes de notifications** ou encore les **applications en temps réel**.

a. Description de l'Observer Pattern

L'**Observer Pattern** fonctionne un peu comme une **chaîne d'informations en direct** : lorsqu'un événement survient, tous ceux qui y sont abonnés reçoivent instantanément la mise à jour. Par exemple, dans une application météo, un changement de température est immédiatement communiqué aux utilisateurs sans qu'ils aient à vérifier manuellement les nouvelles données. Ce modèle permet donc un **découplage** entre les composants (*Cela signifie que les modules sont indépendants mais interagissent grâce au système mis en place*), rendant le programme plus **modulaire et évolutif**. Il offre l'avantage d'éviter un code trop complexe, tout en garantissant une **mise à jour automatique et efficace** des informations. Le patron permet aussi une gestion simplifiée d'observateurs multiples sur un même objet observable. Chaque observateur concret est chargé de faire les mises à jour adéquates en fonction des changements notifiés. Ainsi, l'observé n'est pas responsable des changements qu'il entraîne sur

les observateurs. Son adoption dépend du contexte du projet, mais il reste l'un des modèles les plus prisés pour créer des logiciels réactifs et bien structurés.

b. Diagramme UML du Observer Design Pattern



À titre d'exemple, une [classe](#) (un objet) produit des signaux (données observables), visualisée à travers les panneaux (**observateurs**) d'une [interface graphique](#). La mise à jour d'un signal modifie le panneau qui l'affiche. Afin d'éviter l'utilisation de [thread](#) (suites d'instruction exécutable) ou encore d'inclure la notion de panneau dans les signaux, il suffit d'utiliser le [patron de conception](#) **observateur**.

Le principe est que chaque **classe observable** contient une liste d'**observateurs** ; ainsi, à l'aide d'une méthode de notification, l'ensemble des **observateurs** est prévenu. La classe observée hérite de l'Observable qui gère la liste des **observateurs**. La classe **Observateur** est quant à elle purement [abstraite](#), la fonction de mise à jour ne pouvant être définie que par une classe spécialisée.

c. Points clés du Observer Design Pattern

L'**Observer Design Pattern** offre plusieurs avantages, notamment un **découplage** entre les objets, permettant une communication fluide sans créer de dépendances rigides. Il offre également une **grande flexibilité**, facilitant l'ajout ou le retrait d'observateurs en fonction des

besoins du système, et simplifie la **gestion des événements**, évitant ainsi un code complexe et difficile à maintenir. Cependant, ce modèle présente aussi des inconvénients : s'il n'est pas bien géré, il peut entraîner des **fuites de mémoire**, notamment lorsque les observateurs restent enregistrés inutilement, et sa **complexité augmente** considérablement lorsque le nombre d'observateurs devient trop élevé, ce qui peut impacter la performance du système.

L'Observer **Design Pattern** se distingue des autres **design patterns comportementaux** par son mécanisme de mise à jour automatique des objets observateurs. Cependant, il partage des similarités avec plusieurs autres modèles :

- **Mediator Pattern** (*patron comportemental, fondamental*) : Contrairement à l'**Observer Pattern**, qui permet une mise à jour directe entre le sujet et ses observateurs, le **Mediator** centralise la communication entre plusieurs objets pour éviter les dépendances directes. Il est plus adapté aux systèmes complexes où de nombreux objets doivent interagir sans se connaître directement.
- **Publisher-Subscriber Pattern** (*patron comportemental, non fondamental, dérivé du patron observateur*) : Ce modèle est similaire à l'**Observer Pattern**, mais il fonctionne avec un **système d'événements** où les abonnés sont informés via un intermédiaire (le broker). Contrairement à l'**Observer Pattern**, il permet un découplage encore plus fort, où le sujet n'a pas besoin de connaître ses abonnés.
- **Singleton Pattern** (*patron créateur, fondamental*) : Bien qu'il ne soit pas directement lié à l'**Observer Pattern**, le **Singleton** est souvent utilisé en complément pour garantir qu'un objet sujet (Observable) reste unique dans tout le système.

Ainsi, l'**Observer Pattern** est particulièrement utile lorsque plusieurs objets doivent réagir dynamiquement aux changements d'un sujet, tandis que d'autres modèles offrent des solutions plus adaptées à la gestion d'interactions complexes ou indirectes. Le choix du **design pattern** dépend donc du contexte et des besoins du projet.

II. Analyse SWOT et FAST du projet

1. Analyse SWOT (diagramme)

Forces (Strengths)

- Faible couplage entre les composants : facilite la maintenance et l'extension
- Réactivité en temps réel grâce au mécanisme de fixation automatique
- Architecture modulaire : meilleure gestion des évènements (Java, Python, etc...)

Faiblesse (Weaknesses)

- Peut engendrer des fuites de mémoire si les observateurs ne sont pas bien gérés
- Complexité croissante avec un grand nombre d'observateur
- Dépendance à un modèle synchrone (dans sa version simple) sans web socket
- Nécessite une gestion fine des abonnements/désabonnements

Opportunités (Opportunities)

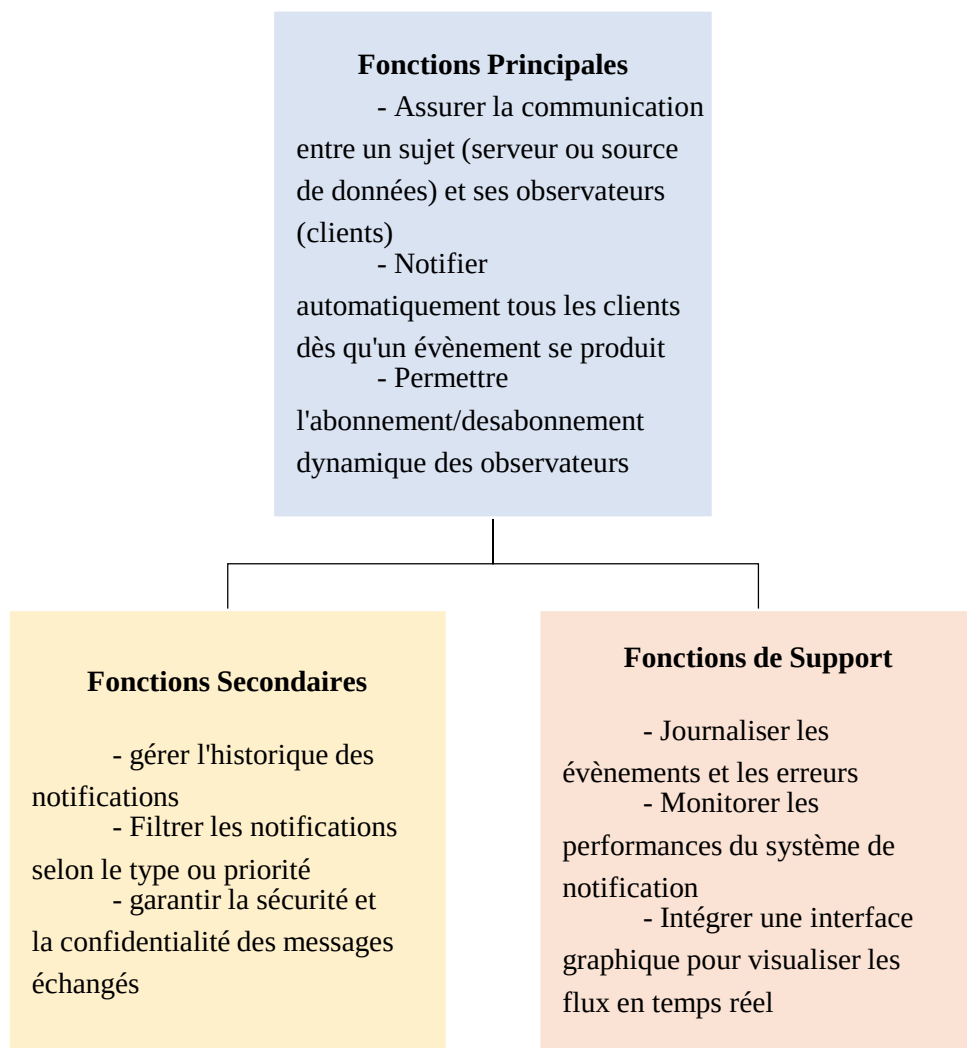
- Forte pertinence dans les applications modernes (chat, monitoring, IoT)
- Possibilité d'intégrer des technologies temps réel comme web socket ou firebase
- Utilisation dans les architectures évènementielles ou orienté micro service
- Extension à des systèmes de notification multiplateforme (email, push, sms)

Menaces (Threats)

- Risque de surcharge des serveurs si les notifications ne sont pas optimisées
- Concurrence avec les systèmes modernes comme Kafka ou MQTT
- Vulnérabilité potentielle si les notifications ne sont pas sécurisées

2. Analyse FAST (diagramme)

FAST (Function Analysis System Technique) permet de décomposer le système selon ses fonctions.



III. Application de l'Observer Design Pattern dans une Messagerie Instantanée

Dans cette section, nous expliquons comment **l'Observer Design Pattern** peut être **appliqué efficacement dans une application de messagerie instantanée**, en détaillant son rôle et les bénéfices qu'il apporte dans la gestion des notifications et des mises à jour en temps réel.

1. Contexte et justification

Les applications de **messagerie instantanée**, comme WhatsApp ou Slack, nécessitent un mécanisme permettant aux utilisateurs de **recevoir des messages en temps réel** sans avoir à les rechercher manuellement. L'**Observer Pattern** permet de gérer cette mise à jour automatique en reliant **les utilisateurs (observateurs)** au **serveur de messagerie (sujet)**. Dès qu'un nouveau message est envoyé, tous les observateurs concernés sont **immédiatement notifiés**.

2. Structure et implémentation dans une application JavaScript

- Le **sujet** (Observable) pourrait être un **serveur de messagerie** qui stocke les conversations et notifie les utilisateurs en cas de nouveaux messages.
- Les **observateurs** seraient les **clients utilisateurs** qui reçoivent automatiquement les mises à jour sans intervention manuelle.
- Lorsqu'un **nouveau message est envoyé**, le serveur **notifie les observateurs inscrits** à une conversation spécifique, leur permettant de **voir le message instantanément**.

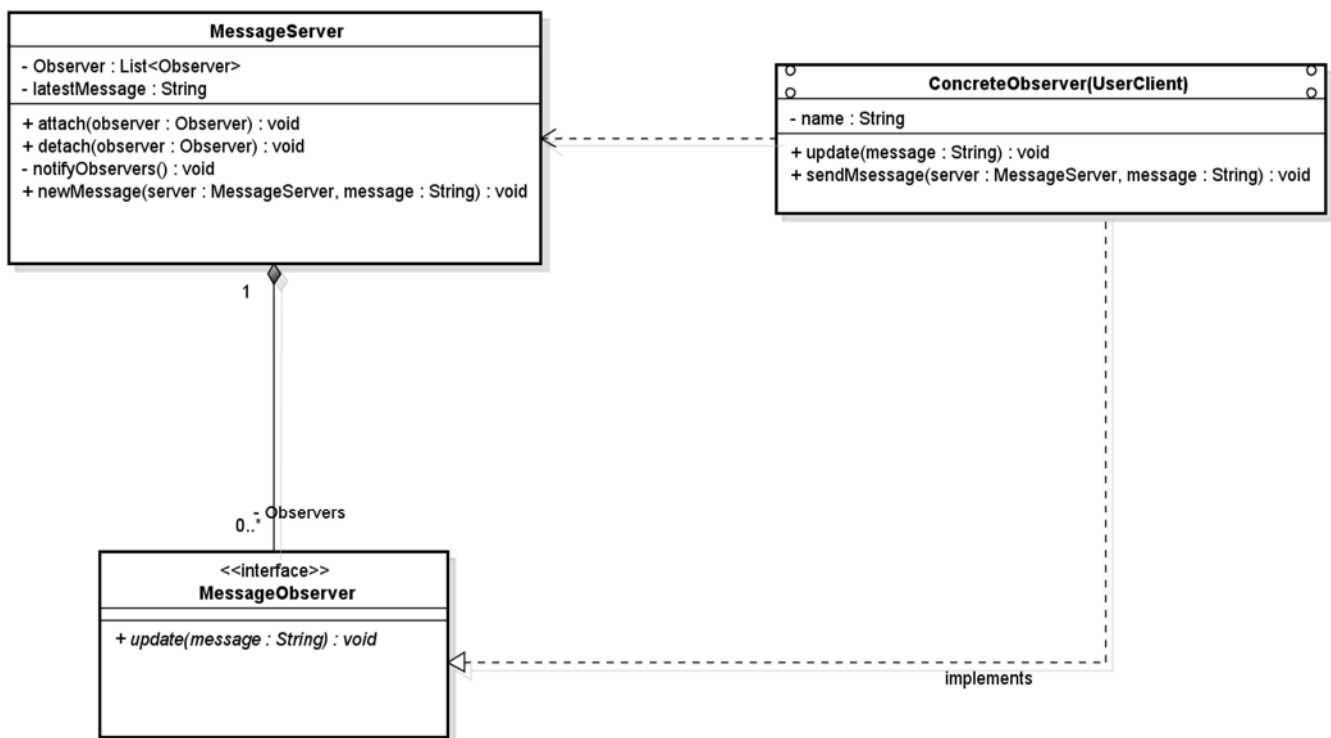
3. Bénéfices pour une messagerie instantanée

- ✓ **Réactivité** : Les utilisateurs reçoivent leurs messages sans délai.
- ✓ **Optimisation des ressources** : Évite les requêtes répétitives de mise à jour, réduisant la charge sur le serveur.
- ✓ **Extensibilité** : De nouveaux utilisateurs peuvent être ajoutés facilement sans modifier le fonctionnement du système.

4. Exemple d'implémentation en JavaScript

Un exemple concret pourrait illustrer la **création d'un serveur de messagerie** utilisant l'**Observer Pattern** avec une classe `MessageServer` jouant le rôle de **sujet**, et des classes `UserClient` agissant comme **observateurs**.

a. Diagramme UML de l'implémentation du design pattern Observer avec JavaScript



- Définition des **rôles des composants** (Sujet, Observateurs, Serveur, Client).

* Sujet (Observable) ou `MessageServer` : "Le diffuseur" il Gère la liste des clients connectés et les notifie quand un nouveau message arrive. C'est le serveur de messagerie, responsable de la gestion des conversations et des messages.

*Observateurs ou `UserClient` : "Les récepteurs", Chaque utilisateur qui écoute les notifications. Ce sont les clients/utilisateurs connectés. Chaque utilisateur abonné à une conversation devient un observateur des événements de cette conversation.

*Serveur (Backend) : "Le pont" Reçoit les messages des clients et les transmet au `MessageServer`.

*Client (Frontend) ou "L'interface utilisateur" : Envoi les messages et les affiche.

IV. Mporôl

Mporôl est une application de messagerie instantanée, conçue pour offrir une communication sécurisée et fluide entre utilisateurs au sein de systèmes variés. Notre objectif principal est de démocratiser l'accès aux outils de messagerie tout en garantissant un contrôle total sur les données personnelles de chaque utilisateur.

L'idée c'est qu'à travers notre API intégrable dans vos différents systèmes (*code source, etc.*), Mporôl vous offre la possibilité de déployer notre messagerie instantanée où que vous le souhaitiez, garantissant ainsi une intégration fluide et personnalisée dans votre environnement. Dans un monde où la protection des données est devenue une préoccupation majeure, Mporôl se distingue par son engagement indéfectible envers la confidentialité. Avec nous, fini les ambiguïtés concernant l'utilisation de vos informations personnelles !

Vous êtes aux commandes : vos données ne sont pas seulement protégées, elles sont respectées. Grâce à notre solution, vous pouvez communiquer en toute sérénité, sachant que votre vie privée est notre priorité absolue.

IV-I. Architecture de l'application

- Explication du **Observer Pattern** appliqué à la messagerie instantanée :

Dans l'application Mporôl, l'architecture repose sur l'utilisation du Observer Design Pattern pour assurer la communication en temps réel entre les utilisateurs. Cette structure garantit que chaque utilisateur connecté à une conversation reçoive instantanément les nouveaux messages, sans avoir à actualiser manuellement l'interface.

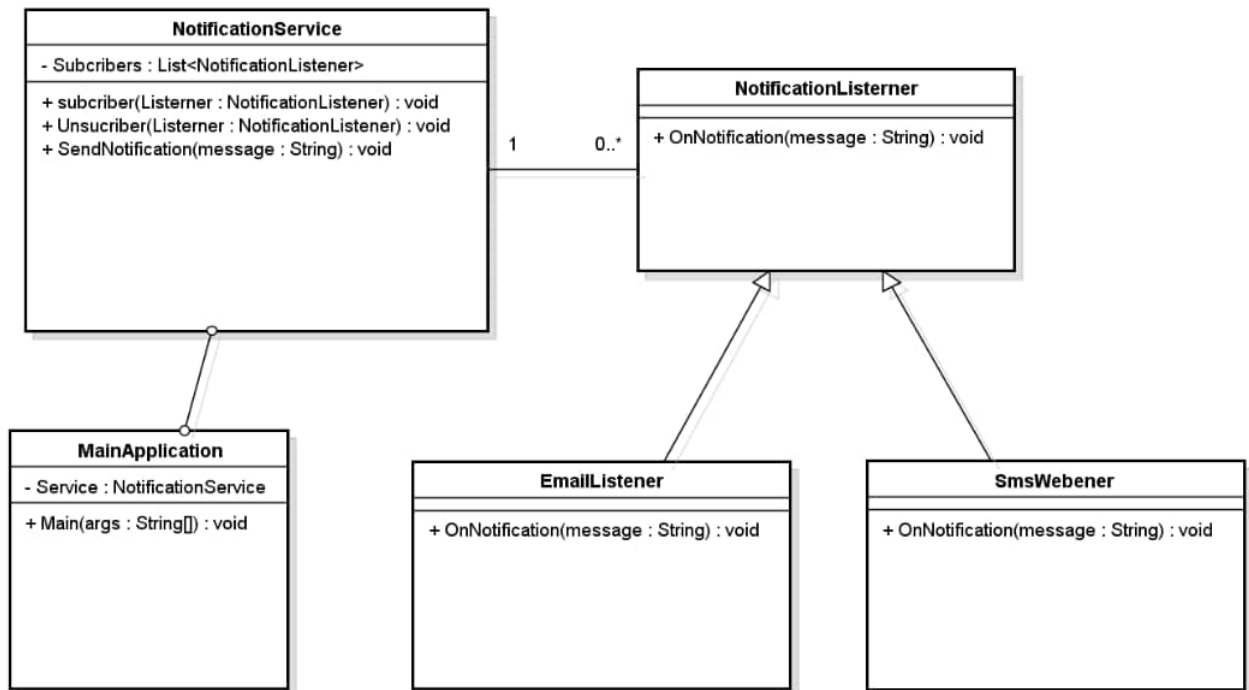
Le rôle du patron observateur ici est d'orchestrer les échanges entre :

Le Sujet (Observable) : c'est le serveur de messagerie, responsable de la gestion des conversations et des messages.

Les Observateurs : ce sont les clients/utilisateurs connectés. Chaque utilisateur abonné à une conversation devient un observateur des événements de cette conversation. Lorsqu'un utilisateur envoie un message, le serveur notifie automatiquement tous les autres utilisateurs

abonnés à la même conversation, qui voient alors apparaître le message en temps réel dans leur interface.

- Diagramme UML illustrant la structure du système :



- Définition des rôles des composants du diagramme :

Un objet (le **NotificationService**) notifie automatiquement ses abonnés (les **Listeners**) quand un événement se produit.

Composants clés :

- **NotificationService** : gère les abonnés et envoie les notifications.
- **NotificationListener** : interface que chaque abonné doit implémenter.
- **EmailListener** / **SmsWebener** : abonnés concrets qui réagissent aux notifications.
- **MainApplication** : point d'entrée, où les abonnés sont enregistrés et les messages déclenchés.

IV-II. Fonctionnalités de Mporôl

- Envoi et réception de messages en temps réel.
- Gestion des **notifications instantanées** pour les utilisateurs observateurs.
- Gestion des sessions avec de nouvelles instances

IV-III Implémentation du Observer Design Pattern

- Explication du fonctionnement du système en temps réel.

Grâce à l'Observer Design Pattern : Chaque participant d'une conversation agit comme un observateur. Dès qu'un utilisateur envoie un message : le serveur (Sujet) enregistre le message, puis notifie tous les observateurs (autres utilisateurs) en temps réel. Aucun rechargement de page n'est nécessaire : le message s'affiche instantanément dans l'interface de chat.

- Explication du **code source** clé.

```
// ===== Fonction d'ajout de message =====
function addMessage({ from, message, sent }) {
  const container = document.getElementById('messages');

  // Retirer le message vide si présent
  const emptyState = container.querySelector('.empty-state');
  if (emptyState) {
    emptyState.remove();
  }

  const msg = document.createElement('div');
  msg.className = 'message ' + (sent ? 'sent' : 'received') + ' time';
  msg.setAttribute('data-time', getcurrentTime());
  msg.innerHTML = `<span class="time">${getcurrentTime()}</span><span class="message-content">${from}: ${message}</span>`;

  container.appendChild(msg);
  container.scrollTop = container.scrollHeight;
}

// ===== PATTERN OBSERVER (NotificationManager) =====

class NotificationManager {
  constructor() {
    this.observers = [];
  }

  subscribe(fn) {
    this.observers.push(fn);
  }

  notify(data) {
    this.observers.forEach(fn => fn(data));
  }
}

const notificationManager = new NotificationManager();

// Abonner une fonction qui affiche une alerte
notificationManager.subscribe(({ from }) => {
  console.log('🚨 Nouveau message de ${from}');
});

// Abonner une fonction qui ajoute le message au DOM
notificationManager.subscribe((data) => {
  addMessage(data);
});
```

Figure 1 - code source observateur

Ici il s'agit du système de notification de notre observer design pattern. NotificationManager est notre Observateur et le champ de la messagerie constitue notre observable. Lorsqu'un message est envoyé le système de notification réagit car l'observable a changé d'état (donc il a reçu une modification dans sa structure globale).

- Exemple de **mise à jour des observateurs**.

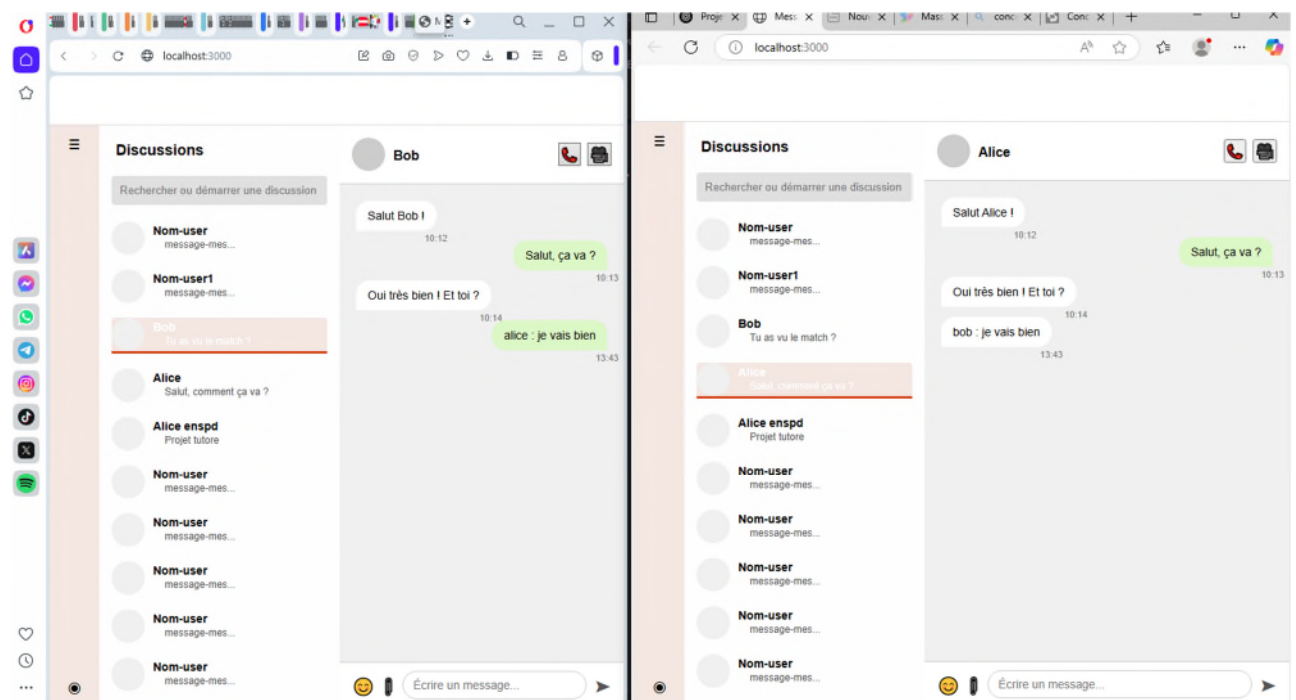


Figure 2 - mise en évidence de la mise à jour

Dans cette image nous avons deux sessions, ici il est mis en évidence à travers le dernier message, les changements d'états encourus par l'interface.

- Gestion efficace des **événements et des notifications**.

Ici on met en évidence l'activation du NotificationManager à travers une alerte¹ « nouveau message » sur une interface utilisateur.

¹ Sur le code actuel c'est une console.log, avant cette fonction renvoyait une "Alert"

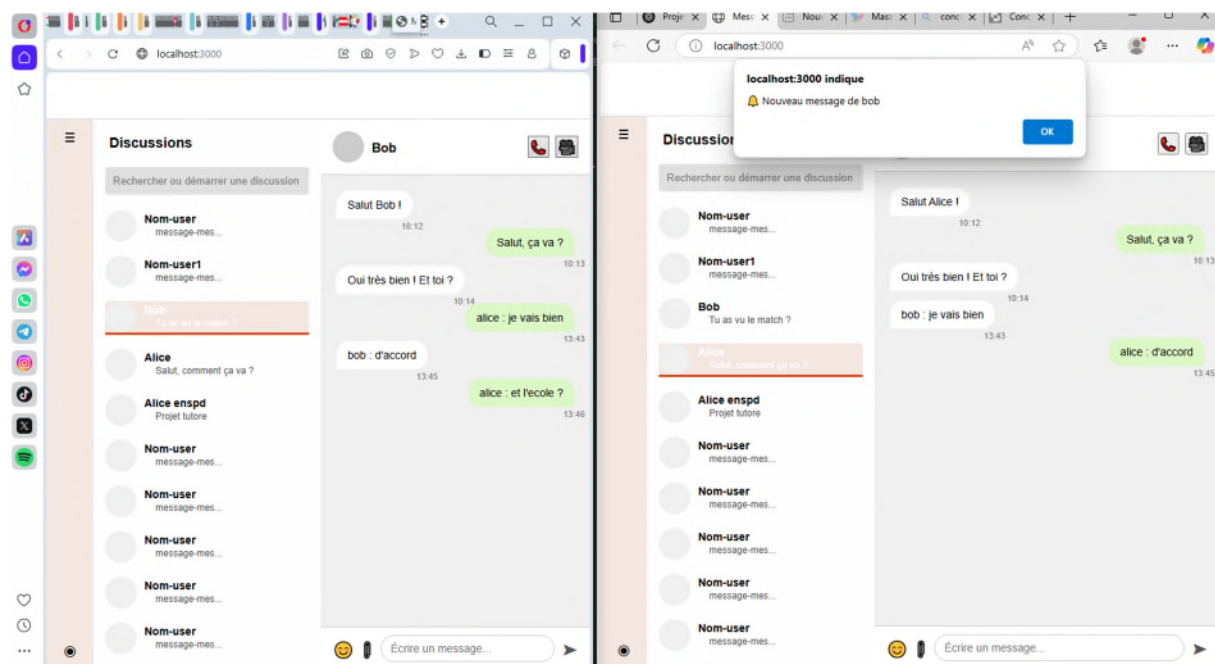


Figure 3 - mise en évidence des notifications

Lorsqu'on ajoute les Web Sockets + Express, La messagerie devient plus dynamique et les messages sont transmis en temps réel sans recharger la page. Le rendu coté serveur donne ce résultat :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
○ PS C:\Users\User\Desktop\observer_pattern>
○ PS C:\Users\User\Desktop\observer_pattern>
○ PS C:\Users\User\Desktop\observer_pattern> npm start

> observer-pattern-chat@1.0.0 start C:\Users\User\Desktop\observer_pattern
Nouvelle connexion: kmTFYyDy8z1vmUYKAAAD
Message reçu: { from: 'User-7dks8', message: 'heyaaaa', sent: true }
Message reçu: { from: 'User-w25x1', message: 'how ?', sent: true }
Message reçu: { from: 'User-w25x1', message: 'çava ou quoi ?', sent: true }
Nouvelle connexion: RzRjCKytvWDP5rxAAAF
Message reçu: { from: 'User-k8ohg', message: 'bande de nazeeeeeee', sent: true }
Message reçu: { from: 'User-w25x1', message: "d'ou il sort ?", sent: true }

```

NB : Le code source Originel de l'application s'est malheureusement perdu mais heureusement nous avons encore le code javascript du design pattern, donc nous avons refait un petit code avec les corrections de l'IA-Claude pour mettre en évidence cette fonctionnalité avec les technologies ajoutées (socket.io). Retrouvez le Template [ici](#).

IV-IV Optimisation et amélioration

- Stratégies pour améliorer les performances de la messagerie.

Pour garantir une expérience utilisateur fluide, notamment en cas d'augmentation du nombre d'utilisateurs ou de messages échangés, plusieurs techniques peuvent être mises en place. Mise en cache des messages récents : en utilisant des technologies comme Redis pour éviter de solliciter la base de données à chaque chargement. Compression des données échangées : minification des fichiers JS/CSS et compression GZIP des réponses HTTP. Pagination ou chargement progressif : les messages anciens sont chargés à la demande (lazy loading), réduisant la charge initiale. Optimisation SQL : création d'index sur les colonnes fréquemment utilisées (ex : conversation_id, timestamp).

- Gestion avancée des **fuites de mémoire** liées aux observateurs.

L'implémentation du Observer Pattern peut engendrer des fuites de mémoire si les observateurs ne sont pas correctement désinscrits après leur cycle de vie.

Solutions proposées :

- **Nettoyage automatique** : prévoir une désinscription (removeObserver()) des clients lorsqu'ils ferment la session ou quittent une conversation.
- **Utilisation de WeakReference (Java)** : pour éviter que le serveur ne conserve une référence forte sur les observateurs inactifs.
- **Journalisation des abonnements** : traquer les ajouts/suppressions d'observateurs pour détecter les anomalies.
- Possibilités d'évolution et de mise à jour du système.

Ajout de fonctionnalités avancées :

Réactions aux messages (like, emoji), réponses en fil de discussion, messages vocaux ou fichiers joints dans le chat.

Amélioration de l'accessibilité :

Thèmes sombres/clairs, support multilingue pour l'interface.

Conclusion

L'**Observer Design Pattern** est un outil puissant qui améliore la conception des systèmes en favorisant une **communication flexible et dynamique** entre objets. Grâce à son mécanisme de **notification automatique**, il permet de maintenir un **découplage efficace** entre les composants, facilitant ainsi la scalabilité et l'évolution des applications. Son adoption est particulièrement pertinente dans des environnements où la mise à jour instantanée des données est essentielle, tels que les **applications de messagerie instantanée**, les **interfaces utilisateur interactives** et les **systèmes de surveillance en temps réel**.

Cependant, bien que ce modèle apporte une structure robuste et adaptable, il **requiert une gestion rigoureuse** des observateurs pour éviter les fuites de mémoire et les problèmes de surcharge liés à une trop grande quantité d'abonnés. Son implémentation doit être soigneusement pensée en fonction des besoins spécifiques du projet, assurant un équilibre entre **performance, maintenabilité et évolutivité**. En définitive, l'**Observer Design Pattern** demeure une approche incontournable pour optimiser la gestion des événements et garantir une application réactive et bien organisée.

Références

1. Gamma, Helm, Johnson, Vlissides - *Design Patterns: Elements of Reusable Object-Oriented Software*
2. Documentation Python / Java
3. [Observateur \(patron de conception\) — Wikipédia](#)
4. [Observer pattern - Wikipedia](#)
5. [Observer Design Pattern | GeeksforGeeks](#)